KUBERNETES

1. Kubernetes Fundamentals

What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to deploy, scale, expose, and manage containerized applications. It continuously compares the desired state declared by the user with the actual cluster state and takes corrective action.

Why Kubernetes?

- Automates deployment and rollout of containerized applications.
- Maintains desired replica counts and replaces failed Pods.
- Provides service discovery and stable networking.
- Supports self-healing, scaling and controlled rollouts.
- Separates application configuration from container images.
- Provides scheduling, resource management, storage and security primitives.

Core mental model

```
Developer / CI pipeline
        |
        v
   Kubernetes API
        |
        +--> Desired state stored in etcd
        |
        +--> Scheduler places Pods
        |
        +--> Controllers reconcile state
        |
        v
   Worker Nodes
   +---------------------------+
   | kubelet + runtime         |
   | kube-proxy / networking   |
   | Pods -> Containers        |
   +---------------------------+
```

Kubernetes object hierarchy

Layer	Object	Interview mental model
Application	Deployment / StatefulSet / DaemonSet / Job	Defines how workload should run
Replica management	ReplicaSet	Maintains desired number of Pods
Runtime unit	Pod	Smallest scheduled/deployable unit
Networking	Service / Ingress	Stable access and HTTP routing
Configuration	ConfigMap / Secret	Externalize configuration and sensitive values
Storage	PV / PVC / StorageClass	Persistent storage lifecycle

2. Kubernetes Architecture

Control Plane

Component	What it does	Interview phrase
kube-apiserver	Exposes the Kubernetes API and is the main entry point for cluster operations.	Front door of the cluster
etcd	Consistent key-value store containing Kubernetes cluster state.	Source of truth for cluster state
kube-scheduler	Finds suitable nodes for unscheduled Pods.	Chooses placement
kube-controller-manager	Runs controllers that reconcile desired vs actual state.	Reconciliation engine
cloud-controller-manager	Integrates Kubernetes with cloud-provider resources when used.	Cloud integration layer

Worker Node components

Component	Role
kubelet	Node agent. Ensures containers described by PodSpecs are running and healthy on the node.
Container runtime	Runs containers. Kubernetes commonly uses CRI-compatible runtimes such as containerd or CRI-O.
kube-proxy	Implements Service networking rules on nodes; exact behavior depends on cluster/network configuration.
CNI plugin	Provides Pod networking and often implements NetworkPolicy capabilities.

Important correction from handwritten notes: A node is not just 'kubelet + kube-proxy + container runtime' in every practical design. CNI networking, DNS and other node agents/add-ons may also be present. The three are the core node concepts to remember for interviews.

3. Pods, Containers and Nodes

Pod

A Pod is the smallest unit Kubernetes schedules. A Pod can contain one or more containers. Containers in the same Pod share the Pod network namespace and can share volumes.

- Most application Pods contain one main application container.
- Sidecars are used when a tightly coupled helper must share the Pod lifecycle/network/storage.
- Pod IPs are ephemeral; use a Service for stable application access.
- A Pod is not normally the unit you scale manually in production; scale the workload controller.

Node

A Node is a physical or virtual machine that provides CPU, memory, networking and storage for Pods. The control plane schedules workloads onto eligible worker nodes.

Pod networking

- Each Pod normally receives a unique cluster IP from the cluster network.
- Containers in the same Pod share the Pod IP and port namespace.
- Pod-to-Pod communication is enabled by the cluster's CNI networking.
- Do not hard-code Pod IPs for application discovery; use Services/DNS.

4. Workload Controllers

Object	Use it when	Key interview point
Deployment	Stateless applications / interchangeable replicas	Manages ReplicaSets and rolling updates
ReplicaSet	Maintain N replicas	Usually managed by a Deployment
StatefulSet	Stable identity/storage is required	Stable ordinal identity + per-Pod storage
DaemonSet	One Pod per eligible node	Agents such as logging/monitoring
Job	Run to completion	Batch work

CronJob	Run Jobs on a schedule	Scheduled batch work

Deployment → ReplicaSet → Pod

```
Deployment
   |
   +--> ReplicaSet
          |
          +--> Pod
          +--> Pod
          +--> Pod

Deployment controls rollout.
ReplicaSet maintains replica count.
Pod runs the containers.
```

Deployment rollout commands

```
kubectl get deployments
kubectl get rs
kubectl get pods
kubectl rollout status deployment/go-web-app
kubectl rollout history deployment/go-web-app
kubectl rollout restart deployment/go-web-app
kubectl rollout undo deployment/go-web-app
kubectl scale deployment/go-web-app --replicas=3
```

5. Kubernetes YAML / Manifest Structure

Most Kubernetes manifests are YAML documents containing the desired configuration of an API object.

The common four fields

Field	Meaning
apiVersion	API group/version for the resource
kind	Resource type such as Deployment, Service or Ingress
metadata	Name, namespace, labels, annotations and object identity
spec	Desired configuration for the object
status	Observed state maintained by Kubernetes; normally do not hand-edit it

Deployment manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-web-app
  labels:
    app: go-web-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: go-web-app
  template:
    metadata:
      labels:
        app: go-web-app
    spec:
      containers:
        - name: go-web-app
          image: myrepo/go-web-app:1.0
          ports:
            - containerPort: 8080
```

Critical selector rule: Deployment spec.selector.matchLabels must match the labels on the Pod template. Service selectors must also match the labels of the Pods they should expose.

Labels and selectors

```
Pod labels:
  app: go-web-app
  env: prod

Service selector:
  app: go-web-app

The Service selects Pods whose labels match its selector.
```

- Labels identify/classify objects.
- Selectors find a set of objects based on labels.
- Services use selectors to choose backend Pods.
- Deployments use selectors to associate ReplicaSets/Pods with the Deployment.

6. Services and Service Discovery

A Service provides a stable network identity for a changing set of Pods. Pods can be recreated and receive new IP addresses, while the Service remains the stable access point.

Service functions

- Provides a stable virtual IP/DNS identity.
- Distributes traffic across eligible backend endpoints.
- Decouples clients from ephemeral Pod IPs.

Service types

Type	Typical use
ClusterIP	Internal cluster access; default Service type.
NodePort	Expose a Service through a port on nodes; useful for simple external/testing access.
LoadBalancer	Request an external load balancer from a supported environment/cloud provider.

ExternalName	DNS alias behavior for an external name; not a normal selector-based Service.

Service YAML

```
apiVersion: v1
kind: Service
metadata:
  name: go-web-app
spec:
  type: ClusterIP
  selector:
    app: go-web-app
  ports:
    - port: 80
      targetPort: 8080
      protocol: TCP
```

Interview trap: port is the Service port. targetPort is the port where the selected Pod/application is listening. A common failure is using the wrong targetPort or a selector that matches no Pods.

Useful Service troubleshooting

```
kubectl get svc
kubectl describe svc go-web-app
kubectl get endpointslices
kubectl get pods -l app=go-web-app -o wide
kubectl describe pod <pod>
kubectl get events --sort-by=.lastTimestamp
```

7. Ingress and External Traffic

Ingress is a Kubernetes API resource for HTTP/HTTPS routing. An Ingress Controller watches Ingress resources and implements the routing using a proxy/load-balancing system.

```
Browser
  |
  v
DNS
  |
  v
External Load Balancer
  |
  v
Ingress Controller
  |
  v
Ingress rules
  |
  v
Service
  |
  v
Pod
```

Ingress vs Ingress Controller

Ingress	Ingress Controller
Declarative routing rules	Actual implementation that watches and enforces those rules
Describes host/path/backend	Usually configures a proxy/load balancer
Does not by itself guarantee an external IP	May create/use a load balancer depending on controller/environment

Ingress example

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: go-web-app
spec:
  ingressClassName: nginx
  rules:
    - host: go-web-app.local
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: go-web-app
                port:
                  number: 80
```

Handwritten-note correction: The Ingress object does not automatically create a load balancer in every Kubernetes environment. The Ingress Controller determines how the rules are implemented. On cloud platforms, the controller may create or configure a cloud load balancer.

Ingress troubleshooting

```
kubectl get ingress
kubectl describe ingress go-web-app
kubectl get pods -n ingress-nginx
kubectl logs -n ingress-nginx <controller-pod>
kubectl get svc -n ingress-nginx
kubectl get endpointslices
```

8. Health Probes

Probe	Question it answers	Effect
Readiness	Can this Pod receive traffic?	Removes Pod from Service endpoints when not ready
Liveness	Is the container still healthy enough to keep running?	Can trigger container restart
Startup	Has this slow-starting application finished initialization?	Delays liveness/readiness checks until startup succeeds

```
readinessProbe:
  httpGet:
    path: /health/ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 10

livenessProbe:
  httpGet:
    path: /health/live
    port: 8080
  periodSeconds: 20

startupProbe:
  httpGet:
    path: /health/startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 5
```

9. ConfigMaps and Secrets

Object	Use	Do not forget
ConfigMap	Non-sensitive configuration	Not a secret store
Secret	Sensitive configuration such as credentials/tokens	Base64 encoding is not encryption

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_ENV: "prod"
  LOG_LEVEL: "info"

---
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  DB_PASSWORD: "change-me"  # example only; do not commit real secrets
```

- Prefer external secret managers for production-sensitive workloads where appropriate.
- Restrict Secret access with RBAC.
- Enable encryption at rest for Secrets where supported/configured.
- Never commit real credentials into Git or container images.

10. Kubernetes Storage

```
Application Pod
     |
     v
PersistentVolumeClaim (PVC)
     |
     v
PersistentVolume (PV)
     |
     v
Storage backend

StorageClass -> enables dynamic provisioning of PVs
```

Concept	Meaning
PV	Cluster storage resource.
PVC	A workload's request for storage.
StorageClass	Defines/provides a storage class and dynamic provisioning behavior.
volumeMounts	Where the container sees the mounted volume.

StatefulSet storage

StatefulSets commonly use volumeClaimTemplates so each ordinal Pod can receive its own PVC. This supports stable identity and storage for workloads such as databases, subject to application/storage design.

11. Scheduling, Resources and Node Management

Requests vs limits

	Requests	Limits

Purpose	Scheduling/capacity planning	Runtime maximum
CPU	Used by scheduler; container can use more if capacity exists and no limit blocks it	CPU is throttled when the limit is enforced
Memory	Used for scheduling	Exceeding memory limit can result in OOM kill

Taints and tolerations

```
kubectl taint nodes worker-1 dedicated=database:NoSchedule

Pod:
tolerations:
  - key: dedicated
    operator: Equal
    value: database
    effect: NoSchedule
```

Affinity / anti-affinity

- Node affinity: choose nodes based on node labels.
- Pod affinity: prefer/require placement near matching Pods.
- Pod anti-affinity: spread workloads away from matching Pods.
- Topology spread constraints can improve distribution across zones/nodes.

Cordon and drain

```
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets
kubectl uncordon <node>
```

Production point: Drain is a controlled eviction process. PDBs, DaemonSets, local storage and other workload characteristics can affect the result. Do not blindly force-delete production workloads.

12. Autoscaling and Availability

HPA

- HPA changes the replica count of a scalable workload.
- It can use CPU/memory or custom/external metrics.
- For CPU utilization targets, resource requests are important because utilization is calculated relative to requests.
- Always check metrics availability when HPA appears stuck.

KEDA

KEDA extends autoscaling around event-driven signals such as queue length, Kafka lag and other external triggers.

PodDisruptionBudget

PDBs protect availability during voluntary disruptions such as maintenance/drain. They are not a replacement for application redundancy or disaster recovery.

13. Kubernetes Networking

Important networking concepts

- Pod IP: ephemeral workload address.
- Service IP/DNS: stable service identity.
- CoreDNS: cluster DNS service.

- CNI: Pod networking implementation.
- kube-proxy: Service forwarding implementation in many cluster configurations.
- NetworkPolicy: traffic allow/deny controls when supported by the CNI.

DNS examples

```
# Service in same namespace
curl http://go-web-app

# Fully qualified Service DNS
curl http://go-web-app.default.svc.cluster.local

# StatefulSet/headless-style discovery can expose individual Pod records,
# depending on the Service and StatefulSet configuration.
```

Troubleshooting DNS: Check CoreDNS Pods/logs, Service/EndpointSlices, CNI networking, NetworkPolicies, node health, resource pressure and upstream DNS configuration.

14. NetworkPolicy

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-app-to-db
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
    - Egress
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: database
      ports:
        - protocol: TCP
          port: 5432
```

Interview answer: first identify the source Pod, destination Pod/namespace, protocol and port; then inspect NetworkPolicies and CNI behavior. Remember that a policy is only effective if the cluster networking implementation supports it.

15. Kubernetes Security

Area	What to remember
RBAC	Roles/ClusterRoles + RoleBindings/ClusterRoleBindings control API permissions.
ServiceAccount	Identity used by workloads to interact with the Kubernetes API and, in EKS, potentially cloud APIs.
Least privilege	Give workloads/users only the permissions they need.
Security context	Run as non-root where possible; restrict Linux capabilities and filesystem behavior.
Secrets	Protect access, encryption and lifecycle; do not treat base64 as encryption.
NetworkPolicy	Limit east-west traffic.
Image security	Use trusted/minimal images, scanning and immutable versions/digests where practical.

```
kubectl auth can-i get pods
kubectl auth can-i create deployments
kubectl get role,rolebinding
kubectl get clusterrole,clusterrolebinding
```

16. Kubernetes Troubleshooting Playbook

A strong interview structure

1. What failed?
2. Where did it fail?
3. When did it start?
4. Check status + events.
5. Check logs.
6. Check configuration/selectors/ports/resources.
7. Check dependencies/networking.
8. Fix root cause.
9. Validate.
10. Prevent recurrence.

Symptom	First checks
CrashLoopBackOff	kubectl logs, kubectl logs --previous, describe, events, probes, env/config, resources
Pending	describe/events, requests, node capacity, taints, affinity, PVC
ImagePullBackOff	image/tag, registry, credentials, network/DNS, permissions
Service has no endpoints	Service selector vs Pod labels, Pod readiness, EndpointSlices
Ingress 502/503	Ingress/controller, Service port, targetPort, endpoints, readiness, app port
Pod cannot reach DB	DNS, NetworkPolicy, Service/endpoints, port, DB listener
Node NotReady	node conditions, kubelet, runtime, disk, memory, CNI, network
HPA not scaling	metrics, requests, target, min/max, HPA events

17. Essential kubectl Commands

Command	Purpose
kubectl get nodes	List nodes
kubectl get pods -A	List Pods across namespaces
kubectl get pods -o wide	Include node/IP information
kubectl get svc	List Services
kubectl get ingress	List Ingress objects
kubectl get deploy,rs,pods	See workload hierarchy
kubectl describe pod <pod>	Detailed state/events
kubectl logs <pod>	Container logs
kubectl logs <pod> --previous	Previous container instance logs
kubectl exec -it <pod> -- /bin/sh	Open a shell when supported
kubectl apply -f file.yaml	Create/update from manifest
kubectl delete -f file.yaml	Delete resources from manifest
kubectl get events --sort-by=.lastTimestamp	Review recent events
kubectl top pods / kubectl top nodes	Resource metrics when metrics API is available

18. Minikube vs EKS

Minikube

Minikube is a local Kubernetes environment useful for learning and testing. It can run Kubernetes using a VM or container-based driver depending on the environment.

```
minikube start
kubectl get nodes
kubectl get pods -A
minikube status
minikube stop
minikube delete
```

EKS

Amazon EKS is a managed Kubernetes service. AWS manages the Kubernetes control-plane service; worker capacity can be provided through managed node groups, self-managed nodes or serverless options such as Fargate, depending on design.

```
aws eks update-kubeconfig --region <region> --name <cluster>
kubectl config current-context
kubectl get nodes
kubectl get pods -A
kubectl get svc -A
```

EKS interview flow: IAM authentication/authorization → Kubernetes RBAC → EKS API endpoint → nodes/Pods → AWS integrations such as load balancers, storage and IAM roles for workloads.

19. Helm — Kubernetes Package Management

Helm packages Kubernetes manifests into reusable Charts. A chart contains templates plus default values and metadata. A release is an installed instance of a chart.

Chart structure

```
go-web-app-chart/
├── Chart.yaml
├── values.yaml
├── templates/
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   └── _helpers.tpl
└── charts/
```

Key Helm concepts

Concept	Meaning
Chart.yaml	Chart metadata such as name/version/appVersion/dependencies.
values.yaml	Default inputs to templates.
templates/	Parameterized Kubernetes manifests.
Release	A specific installed instance/version of a chart.
_helpers.tpl	Reusable named template helpers.
helm template	Render templates locally for inspection.

Helm commands

```
helm create go-web-app-chart
helm lint ./go-web-app-chart
helm template go-web-app ./go-web-app-chart
helm install go-web-app ./go-web-app-chart
helm upgrade --install go-web-app ./go-web-app-chart
helm list
helm status go-web-app
helm history go-web-app
helm rollback go-web-app <REVISION>
helm uninstall go-web-app
```

Values example

```
# values.yaml
image:
  repository: myrepo/go-web-app
  tag: "1.0"

replicaCount: 2

service:
  type: ClusterIP
  port: 80

# template usage:
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
replicas: {{ .Values.replicaCount }}
```

Production rule: Do not put real passwords/tokens directly in values.yaml committed to Git. Prefer external secret mechanisms or secure CI/CD injection.

20. Kubernetes CI/CD Integration

A practical DevOps pipeline can connect Git → CI → image build → registry → Helm/Kubernetes deployment → verification.

```
Developer
   |
   v
GitHub / GitLab
   |
   v
CI pipeline
   +--> Build
   +--> Unit tests
   +--> Static/security checks
   +--> Docker image build
   +--> Push image to registry
   |
   v
Update Helm values / image tag
   |
   v
helm upgrade --install
   |
   v
Kubernetes
   +--> Deployment
   +--> Service
   +--> Ingress
   |
   v
Smoke test + rollout verification
```

Example CI stages

Stage	Typical action
1. Build & test	Compile/package application and run unit tests
2. Static/security checks	Lint/SAST/dependency/image checks according to policy
3. Docker image	Build immutable versioned image
4. Registry	Authenticate and push image
5. Helm	Pass image tag and environment values
6. Deploy	helm upgrade --install

7. Verify	rollout status + smoke tests + logs/metrics

Deployment principle: Promote the same tested artifact across environments instead of rebuilding different artifacts for each environment.

21. Deployment Strategies

Strategy	How it works	Rollback idea
Rolling	Gradually replace old Pods with new Pods.	Rollback Deployment revision
Blue-Green	Run old and new environments together; switch traffic.	Switch traffic back
Canary	Send a small percentage/segment of traffic to new version first.	Reduce/remove canary traffic

22. Production EKS Design Checklist

- Use appropriate public/private API endpoint controls.
- Use IAM least privilege and Kubernetes RBAC.
- Use IAM roles for workloads where appropriate instead of long-lived cloud credentials.
- Use managed node groups or another deliberate node lifecycle strategy.
- Separate workloads with namespaces and policies.
- Use resource requests/limits and quotas.
- Use NetworkPolicies where supported and required.
- Use encrypted storage and protected Secrets.
- Use load balancer/controller integrations deliberately.
- Monitor cluster, nodes, Pods, application metrics, logs and events.
- Plan Kubernetes/add-on/node upgrades and validate compatibility.
- Use immutable image versions and controlled CI/CD.